ENUNCIADO DO TRABALHO PRÁTICO (N2)

Disciplina: DESENVOLVIMENTO DE SOFTWARE PARA DISPOSITIVOS MÓVEIS

Tema: Aplicativo Android Full-Stack em Kotlin: Visão Computacional, Inferência On-Device e Persistência em Servidor

---

1 Objetivo do Trabalho

O objetivo deste trabalho prático é consolidar os conhecimentos adquiridos na disciplina através do desenvolvimento de uma solução computacional completa (**Full-Stack Mobile**). Os alunos deverão construir um aplicativo nativo em **Kotlin** focado em **Visão Computacional e Inferência On-Device** integrado a um **Servidor Backend** para persistência de dados e estatísticas.

Os grupos deverão definir a proposta do seu produto móvel, projetar o seu próprio **Design System** (paleta de cores, tipografia e identidade visual), escolher um dataset no **Roboflow**, integrar a inferência local off-line com modelos **YOLO / ONNX** fornecidos pelo professor e comunicar-se com um servidor backend (desenvolvido em Node.js, Python, Kotlin ou linguagem à escolha) para armazenar e consultar o histórico e os relatórios geométricos de cada análise realizada.

—

2 Fluxo de Trabalho e Etapas do Projeto

O desenvolvimento da solução seguirá o seguinte fluxo de trabalho entre os grupos e o professor:

Etapa 1: Seleção do Dataset no Roboflow: O grupo escolhe no **Roboflow Universe** (`https://universe.roboflow.com`) um dataset público rotulado de detecção de objetos adequado ao seu projeto.

Etapa 2: Validação com o Professor: O grupo submete a proposta e o link do dataset para aprovação do professor.

Etapa 3: Treinamento YOLO e Entrega ONNX: O professor realiza o treinamento da rede neural **YOLO** e disponibiliza para a equipe o modelo em formato **ONNX** (`.onnx`).

Etapa 4: Criação do Design System do Grupo: Os alunos elaboram a identidade visual do aplicativo (paleta de cores, tipografia, estilos de botões e cartões).

Etapa 5: Desenvolvimento do Servidor Backend: Construção de uma API REST (em Node.js, Python, Kotlin, etc.) com rotas para receber, persistir e retornar os relatórios de inferência em Banco de Dados (SQL/NoSQL) ou em arquivos JSON.

Etapa 6: Desenvolvimento da Aplicação Android em Kotlin: Construção do aplicativo móvel com inferência local off-line via ONNX Runtime, envio automático da telemetria para o servidor backend e telas de consulta ao histórico de análises.

—

3 Requisitos Funcionais e Arquitetura

Os grupos deverão implementar os seguintes requisitos funcionais no aplicativo e servidor:

RF1: Gerenciamento de Modelos ONNX: Suporte para carregar o modelo ONNX embarcado ou selecionar um arquivo `.onnx` a partir do armazenamento do dispositivo.

RF2: Aquisição de Imagens (Câmera / Galeria): Integração funcional com a câmera nativa e com a galeria de mídia do Android.

RF3: Ajuste Dinâmico de Parâmetros: Controle interativo com *Slider* para o Limiar de Confiança (*Confidence Threshold*) e filtro de rótulos/classes.

RF4: Pipeline de Inferência Local On-Device: Execução 100% local do modelo ONNX no hardware móvel (CPU/NPU/GPU) sem dependência de nuvem.

RF5: Cálculo das Métricas Geométricas: Para cada *Bounding Box* detectado, o app deve calcular:

- Largura (W_{px}) e Altura (H_{px}) da caixa delimitadora em pixels.
- As coordenadas do Centróide da caixa delimitadora em pixels:

$$X_{centroide} = X_{min} + \frac{W_{px}}{2}, \quad Y_{centroide} = Y_{min} + \frac{H_{px}}{2} \tag{1}$$

- Área da caixa delimitadora em pixels² ($A_{px} = W_{px} \times H_{px}$).

RF6: Comunicação com o Servidor Backend (API REST): Transmissão automática via HTTP (POST JSON) dos dados da análise para o servidor backend após a inferência.

RF7: Persistência e Consulta no Servidor: O servidor backend deve salvar no banco de dados ou em arquivos JSON estruturados o total de objetos, data/hora, tempo de execução e o vetor com as dimensões e centróides de cada caixa delimitadora.

RF8: Tela de Histórico e Detalhamento de Inferências:

- Exibição de uma tela com a listagem cronológica de todas as inferências realizadas (com data, hora, modelo e quantidade de objetos).
- Navegação ao clicar em um item da lista para uma tela de detalhamento, onde o usuário poderá consultar individualmente as dimensões e o centróide de cada caixa delimitadora.

RF9: Arquitetura Limpa em Kotlin: Emprego de MVVM, *Coroutines* para assincronismo e manipulação de estado reativo com *StateFlow* / *LiveData*.

4 Estrutura do Payload de Comunicação (JSON)

Após a inferência local, o aplicativo Android deve montar e enviar um payload JSON ao servidor com a seguinte estrutura de referência:

```
{
  "id_sessao": "sessao_20260918_001",
  "data_hora": "2026-09-18T23:06:00Z",
  "nome_modelo": "best.onnx",
  "tempo_execucao_ms": 909,
  "total_objetos": 51,
  "confianca_media": 0.90,
  "dimensao_imagem": { "largura_px": 1080, "altura_px": 1920 },
  "caixas_delimitadoras": [
    {
      "id_caixa": 1,
      "rotulo_classe": "tilapia",
      "confianca": 0.93,
      "largura_px": 120.5,
      "altura_px": 45.0,
      "centroide_x": 210.25,
      "centroide_y": 422.50,
      "area_px2": 5422.5
    },
    {
      "id_caixa": 2,
      "rotulo_classe": "tilapia",
      "confianca": 0.90,
      "largura_px": 115.0,
      "altura_px": 48.0,
      "centroide_x": 350.00,
      "centroide_y": 510.00,
      "area_px2": 5520.0
    }
  ]
}
```

—

5 Diagramas Arquiteturais de Referência

Para orientar a implementação das classes no aplicativo Android e o modelo de banco de dados no servidor backend, o grupo deve consultar os diagramas de referência a seguir:

5.1 Diagrama de Classes UML (App Mobile e Servidor)

O diagrama da Figura 1 detalha o modelo de domínio no Kotlin e a integração com os componentes do servidor backend.

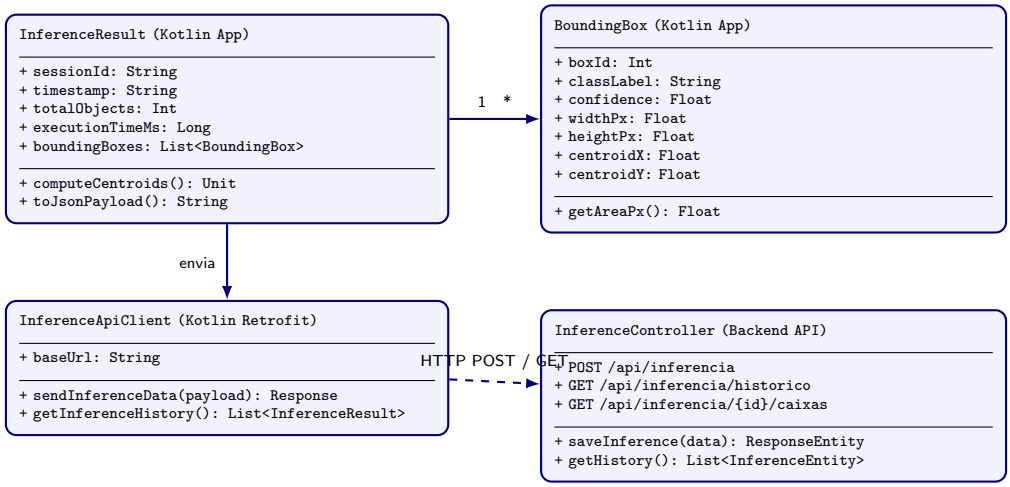


Figura 1: Diagrama de Classes UML: Estruturas de dados no Kotlin e comunicação com o Backend.

5.2 Diagrama Conceitual do Banco de Dados (ER)

O diagrama da Figura 2 representa o modelo conceitual do banco de dados relacional ou NoSQL a ser implementado no servidor backend para persistência dos resultados.

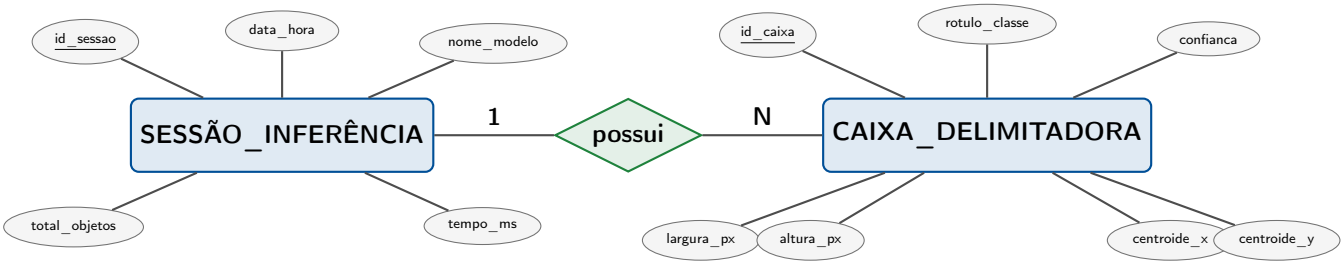


Figura 2: Diagrama Conceitual do Banco de Dados para Persistência das Inferências.

6 Protótipos de Interface Gráfica e Fluxo de Navegação

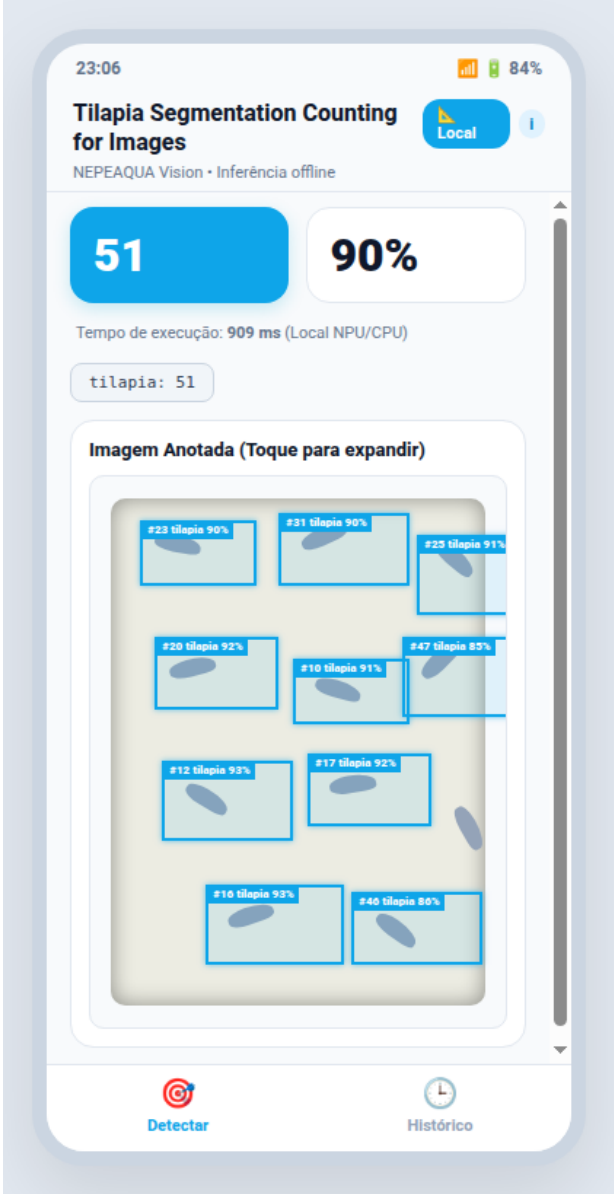
Os protótipos fornecidos a seguir servem como **guia estrutural e de navegação (wireframes)** para o layout e fluxo entre as telas do aplicativo móvel.

6.1 Telas de Execução e Exibição da Inferência (Telas 1 e 2)

As Figuras 3a e 3b ilustram a configuração dos parâmetros, seleção da imagem e o resultado gráfico imediato do processamento.



(a) Tela 1: Configuração e Seleção de Imagem

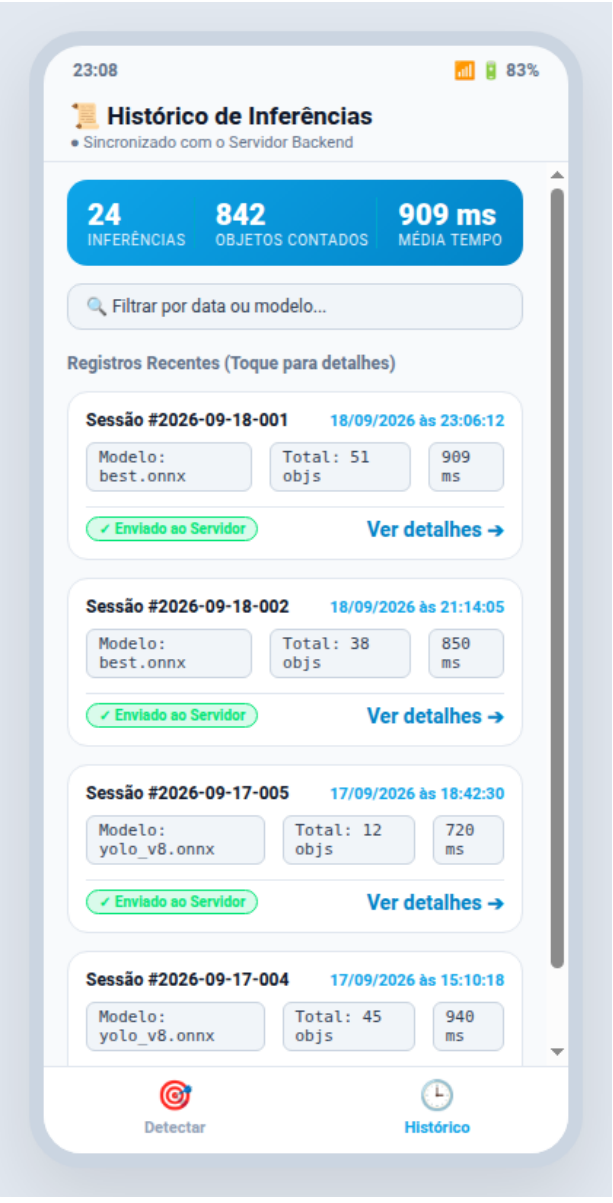


(b) Tela 2: Resultados e Visualização Anotada

Figura 3: Protótipos para a execução da inferência e exibição direta das caixas anotadas.

6.2 Telas de Histórico e Detalhamento das Caixas (Telas 3 e 4)

As Figuras 4a e 4b ilustram o fluxo de consulta onde o usuário visualiza o histórico de análises e acessa os detalhes geométricos de cada caixa delimitadora.



(a) Tela 3: Histórico de Inferências (com Data e Hora)



(b) Tela 4: Detalhamento Geométrico das Caixas

Figura 4: Protótipos para o fluxo de histórico e consulta detalhada das caixas delimitadoras.

—

7 Orientação para Criação do Design System e Telas

Os alunos deverão elaborar o seu próprio Design System, podendo escolher a paleta de cores e tipografia. Contudo, **o aplicativo deve obrigatoriamente seguir o tema Light Mode e ser desenvolvido utilizando exclusivamente a biblioteca Jetpack Compose (não será aceito o uso de XML Views)**.

Além disso, a estrutura de navegação deve contemplar uma **Bottom Navigation Bar** com os itens **Detectar** e **Histórico**, além das seguintes 4 telas:

1. **Tela 1 (Configuração & Entrada):** Escolha do modelo ONNX, opções de Câmera/Galeria e Slider de Limiar de Confiança.
2. **Tela 2 (Resultado Gráfico):** Exibição da contagem total, tempo em ms e imagem anotada com as caixas sobrepostas.
3. **Tela 3 (Histórico de Inferências):** Listagem cronológica de todas as sessões de inferência salvas (exibindo data, hora, quantidade de objetos, modelo utilizado e indicador de envio ao servidor).
4. **Tela 4 (Detalhamento Geométrico das Caixas):** Tela de consulta acionada ao tocar em um item da Tela 3, apresentando a auditoria individual de cada caixa delimitadora (*Bounding Box*):
   - Identificador da caixa (ex: `Caixa #01`).
   - Rótulo da classe e percentual de confiança.
   - Dimensões exatas: Largura (W_{px}) e Altura (H_{px}) em pixels.
   - Coordenadas exatas do Centróide ($X_{centroide}, Y_{centroide}$) em pixels.
   - Área da caixa em pixels² (A_{px}).

—

8 Critérios de Avaliação (Pontuação total: 10,0)

9 Entrega e Informações Gerais

- **Formato:** Trabalho em grupo (até 4 alunos).
- **Submissão:** Links dos repositórios públicos no GitHub (Aplicativo Android + Servidor Backend) e arquivo APK até a data limite estipulada.
- **Linguagens: Kotlin com Jetpack Compose** para o aplicativo Android e linguagem livre para o Servidor Backend (Node.js, Python, Kotlin, etc.).

Critério de Avaliação	Pontuação
1. Seleção do Dataset (Roboflow): Escolha do problema, relevância e submissão prévia para aprovação do professor.	1,0 pt
2. Design System e Interface Móvel: Criação do Design System próprio e fidelidade à estrutura dos wireframes (Telas 1 a 4).	2,0 pts
3. Inferência Local ONNX e Geometria: Integração do modelo YOLO ONNX no Android e cálculo de largura, altura, centróides e área das caixas.	2,5 pts
4. Fluxo de Histórico e Consulta (Telas 3 e 4): Implementação da lista cronológica com data/hora e tela de detalhamento geométrico de cada caixa.	2,0 pts
5. Servidor Backend, Persistência e Git: API REST (Node/Python/Kotlin), persistência em BD/JSON e organização dos repositórios.	1,5 pts
Total	**10,0 pts**